



LLM-Based RESTful Web API Service Discovery

Wen Li^{1,2} · Xin Ai^{1,2} · Guosheng Kang^{1,2} · Xinci Qiu^{1,2} · Jianxun Liu^{1,2}

Received: 5 June 2025 / Revised: 26 September 2025 / Accepted: 8 November 2025
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2025

Abstract

The rapid advancement of service computing has led to an explosion in the number of RESTful Web APIs available online. Although this abundance offers developers greater flexibility, it also poses significant challenges in accurately discovering the required Web APIs that meet specific user queries. Existing Web API discovery methods mainly rely on keyword matching or semantic analysis of API description documents and interfaces. However, these methods often fail to effectively align user queries with the semantic content of Web API descriptions, leading to suboptimal results. To address this issue, we propose an LLM-based Web API discovery framework tailored for the RapidAPI platform. Specifically, we design a three-level prompt strategy, category level, API level, and endpoint level, that progressively narrows the search space to accurately identify relevant APIs and the corresponding endpoints. Additionally, we incorporate a self-checking mechanism to improve the accuracy and reliability of the discovery process. Extensive experiments across diverse development scenarios demonstrate that the proposed method outperforms baseline methods on multiple LLMs. Case studies further validate the effectiveness of the proposed method, highlighting the potential of large language models in advancing Web API service discovery.

Keywords LLM · RESTful Web API · service discovery · RapidAPI · prompt strategy

1 Introduction

With the rise of service-oriented architectures and the advancement of Internet technologies, an increasing number of Web services are published, discovered, and accessed online [1, 2]. As a result, more developers prefer reusing existing services over building them from scratch. Service directories like the Universal Description, Discovery,

and Integration (UDDI) were created, and the Web Service Description Language (WSDL) became the standard for describing SOAP-based services [3]. Following the introduction of the REST architectural style, Web services have gradually evolved into RESTful Web APIs due to their simplicity and lightweight nature [4–6]. Consequently, service registration and discovery platforms have shifted from traditional registries like UDDI to popular API marketplaces such as RapidAPI¹, ProgrammableWeb², and others. This abundance offers developers significant flexibility and reuse opportunities, but it also brings forth a critical challenge: efficiently and accurately discovering the most suitable APIs that match specific development requirements or user queries. We select RapidAPI in our work because it is one of the largest and most diverse API marketplaces, hosting over 40,000 endpoints across multiple domains such as text analysis, translation, weather, and e-commerce. Besides, RapidAPI is widely adopted in real-world software development workflows, where developers frequently search for services via RapidAPI. Although our implementation is tailored to RapidAPI's data structure, the core framework of hierarchical

✉ Guosheng Kang
guoshengkang@gmail.com

Wen Li
wenli0023cs@gmail.com

Xin Ai
dm5201314ya@gmail.com

Xinci Qiu
xinciqiu27@gmail.com

Jianxun Liu
ljx529@gmail.com

¹ Hunan Provincial Key Lab. for Services Computing and Novel Software Technology, Hunan University of Science and Technology, Xiangtan 411201, China

² School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan 411201, China

¹ <https://rapidapi.com/hub>

² <https://www.programmableWeb.com/> (now it is transferred to <https://www.mulesoft.com/programmableWeb>)

prompting and self-checking is generalizable and can be adapted to other platforms like ProgrammableWeb with minor adjustments (e.g., adjusting the organizational structure of data or simplifying our method to a two-level prompt strategy).

The shift in service paradigms has redirected service discovery research from SOAP-based services to RESTful APIs [7, 8]. However, traditional WSDL-based service discovery approaches mainly rely on keyword matching, which often fails to ensure accurate results [9]. To address this limitation, ontology-based semantic description languages were introduced to enhance parameter understanding [10]. However, since RESTful APIs typically lack semantic annotations, these SOAP-oriented methods are difficult to adapt to the API context. Meanwhile, most research on Web API discovery focuses on enhancing natural language processing (NLP) techniques to better interpret textual descriptions [11]. However, due to the sparsity and semantic ambiguity of descriptive documents, the quality of service discovery is compromised. At the same time, the brevity of user queries makes their intent insufficiently clear, which also leads to poor service discovery quality.

Recent advances in large language models (LLMs), such as OpenAI's GPT series [12], have shown great promise in understanding and generating natural language with remarkable fluency and contextual awareness [13]. These capabilities have opened new avenues for improving software engineering tasks, including code generation, documentation synthesis, and service discovery. Motivated by this, we explore how LLMs can be leveraged to address the long-standing challenges in RESTful Web API discovery. In this paper, we propose a novel LLM-based framework for Web API discovery, specifically targeting the RapidAPI platform. Our approach is designed to bridge the gap between user requirement queries and API documentation through a hierarchical prompt-based mechanism. Specifically, we design a three-level prompting strategy—category level, API level, and endpoint level, that progressively narrows the search scope and guides the LLM in locating the most relevant API endpoints. To further enhance the reliability of the results, we incorporate a self-checking mechanism that verifies the consistency and relevance of the selected APIs against the input query. In summary, the contributions of this paper are as follows:

- We propose an LLM-based Web API discovery framework tailored for the RapidAPI platform, effectively aligning user queries with API endpoint documentation.
- We design a hierarchical prompt structure that enables fine-grained API discovery at the category, API, and endpoint levels.
- We introduce a self-checking mechanism to improve the accuracy and reliability of the discovery results.

- We conduct comprehensive experiments and case studies to validate the effectiveness and robustness of our method using multiple LLMs compared to existing baselines.

The remainder of this paper is structured as follows. Section 2 reviews the related work on service discovery. Section 3 outlines the preliminaries used in our work. Section 4 details the proposed method in detail. Section 5 presents the experimental results along with a comprehensive analysis. Finally, Section 6 concludes the paper.

2 Related Work

Early research on service discovery was primarily centered around syntactic and semantic techniques for locating SOAP-based services described using WSDL and registered in UDDI [14, 15]. These methods evolved to incorporate ontology-based reasoning for improved semantic matching. However, the rise of RESTful Web APIs introduced new challenges due to the absence of formal descriptions and reliance on unstructured natural language documentation. In this section, we review prior work on service discovery, focusing on two major paradigms: *SOAP-based Web services* and *RESTful Web APIs*.

2.1 Service Discovery for SOAP-based Web Services

In the early stages of service computing, SOAP-based Web services were the dominant paradigm for distributed service composition and reuse. To facilitate the discovery of such services, the UDDI registry and WSDL were widely adopted standards. Traditional discovery approaches primarily relied on syntactic keyword matching over WSDL files and UDDI metadata [3, 16], which limited their effectiveness due to vocabulary mismatches and lack of semantic understanding. To address these limitations, researchers introduced semantic enhancements through ontology-based methods. Semantic Web Service frameworks such as OWL-S and WSMO incorporated formal semantics to improve service matchmaking accuracy [17, 18]. These methods enabled reasoning over service inputs, outputs, and capabilities using Description Logic and ontology alignment techniques. For example, Paolucci et al. proposed a matchmaking algorithm that leveraged OWL-based descriptions for automated service selection [18]. Later works expanded on these ideas with machine learning and hybrid approaches that combined syntactic and semantic similarity measures [19].

Despite their effectiveness, these techniques are tightly coupled with SOAP-based services and heavily rely on structured semantic annotations. As the service ecosystem transitioned to RESTful architectures, the assumptions and infrastructure underlying these methods became less appli-

cable due to the lightweight, unstructured nature of RESTful Web APIs.

2.2 Service Discovery for RESTful Web APIs

The shift from SOAP to REST introduced new challenges in service discovery, primarily due to the lack of formal service descriptions and semantic annotations. RESTful Web APIs are typically described through natural language documentation, often hosted on platforms like RapidAPI, ProgrammableWeb, or Swagger-based portals [5, 6]. Consequently, discovery methods for RESTful APIs have increasingly relied on NLP techniques to interpret textual metadata and user queries [20].

Specifically, He et al. [21] propose a keyword-based approach that automates system planning, service discovery, and selection to support the construction of service-based systems, enabling engineers to compose Web services using simple keyword queries with quality constraints and optimization goals. Ma et al. [22] propose TASSIC (Test-Oriented API Search with Semantic Interface Compatibility), a novel RESTful Web API/service discovery approach that combines semantic term expansion, interface compatibility analysis, and test-driven validation to improve search accuracy by ensuring semantic relevance, functional correctness, and runtime availability. Liu et al. [11] propose an intelligent Web API search system using a two-step transfer learning approach, leveraging crowdsourced data and ensemble deep learning to match natural language queries with relevant APIs and their endpoints efficiently. Wang et al. [23] propose method is an automated RESTful API service discovery framework that combines functional description texts with interface features—including parameter abbreviation expansion, required/optional input differentiation, and one-to-many parameter matching—to improve both accuracy and callability of discovered services. However, due to the ambiguity and brevity of both API descriptions and user queries, these methods still face significant limitations in capturing user intent accurately.

Recent advances in large language models (LLMs), such as GPT and DeepSeek, have demonstrated remarkable capabilities in natural language understanding and semantic reasoning by leveraging massive amounts of pretraining data and deep neural architectures [13]. These models can capture nuanced meanings, context, and implicit user intent without requiring task-specific labeled data. These capabilities have spurred new research directions in leveraging LLMs for software engineering tasks, including code search, API recommendation, and automated documentation. However, few studies have systematically explored the application of LLMs in hierarchical Web API discovery. Our work distinguishes itself by proposing a structured, prompt-driven LLM-based framework that supports fine-grained discov-

ery across multiple levels (category, API, endpoint), while incorporating a self-verification mechanism to ensure result reliability. In addition to service discovery, a related line of research focuses on Web API service recommendation, which typically leverages Mashup requirements and historical Mashup-API interactions [24]. The emergence of LLMs has also inspired novel recommendation methods. Liu et al. [25] employed LLMs for QoS prediction to improve service recommendation, while Zou et al. [26] integrated LLM-enhanced semantic representations with category co-occurrence features for more accurate Web API recommendations.

3 Preliminaries

In practice, an API service typically consists of multiple functionally related endpoints, with each endpoint representing the fundamental unit for selection and invocation. Accordingly, our approach operates at the endpoint level. Unless stated otherwise, the term “service” throughout this paper specifically refers to an endpoint. The problem formalization primarily involves two components: the user request and the corresponding endpoint service information. In this section, we present an overview of LLM and some key definitions used in our work.

3.1 Overview of LLM

Large Language Models (LLMs) refer to deep learning models trained on extensive text corpora to perform a broad range of NLP tasks. Most state-of-the-art LLMs are built upon the Transformer architecture, which relies on a self-attention mechanism. This mechanism maps input tokens into query, key, and value vectors, and calculates attention scores based on the similarity between queries and keys. The resulting weighted sum of values allows the model to capture token-level interactions across the entire sequence, effectively modeling both local and global dependencies. This enhances the model’s capacity to represent contextual semantics throughout the input, even when queries are short or ambiguous. Furthermore, LLMs demonstrate strong generalization capabilities. Through large-scale pretraining, they acquire extensive syntactic and semantic knowledge, enabling effective performance on various downstream tasks with minimal labeled data. Additionally, LLMs exhibit robust language generation abilities, producing fluent and contextually relevant text, which makes them highly effective for a wide range of NLP applications. Unlike keyword matching or static embedding methods, LLMs can capture subtle semantic nuances that are critical for aligning user needs with API functionalities. Furthermore, through structured prompting, LLMs can perform multi-step reasoning, which enables a

hierarchical approach to discovery and improves the accuracy of matching between user queries and unstructured API documentation.

3.2 Key Definitions

Definition 1 (Category of Web API). On the RapidAPI platform, Web APIs are organized into functional categories that group related services under a shared theme. This categorization is similar to the Java Package mechanism, which provides a logical structure for organizing software components. These categories facilitate efficient navigation and help users quickly locate relevant service modules, serving as the initial level of entry for service discovery. In this context, APIs serve as units of service aggregation, while endpoints represent individual, executable service operations.

Definition 2 (RESTful Web API). This study proposes a hierarchical service discovery method for RESTful Web APIs using large language models (LLMs). The approach leverages both development context and user query intent to perform multilevel reasoning, guiding the discovery process from category to endpoint. The method is applied to the RapidAPI platform, where APIs function not as single callable methods but as aggregates of related functionalities, akin to Java Classes. This abstraction supports more flexible and scalable service composition and discovery.

Definition 3 (Endpoint). On the RapidAPI platform, each API comprises multiple endpoints, with each endpoint corresponding to a specific, callable operation. Endpoints serve as the actual interfaces through which functional tasks are executed. While APIs provide a structural grouping of related capabilities, endpoints implement the concrete service logic. For instance, a weather API may include endpoints for current conditions, forecasts, or historical data. Thus, our discovery method aims to identify endpoints that match user needs rather than stopping at the broader API level.

4 Method

This section introduces the LLM-based hierarchical Web API discovery framework via prompt methodology, which is shown in Fig. 1. The proposed framework consists of three levels in sequence, i.e., category level, API level, and endpoint level. At each level, specific prompts are designed to guide the LLM in inferring the items (i.e., category, API, or endpoint). Meanwhile, a self-checking mechanism is designed into the framework to automatically examine and refine the service discovery results, thereby enhancing the accuracy and reliability of the results.

4.1 Service Discovery in Category Level

In service-based software development, developers often find their required services through user queries. Therefore, accurately identifying the underlying intent behind queries is crucial, as it significantly affects the quality of service discovery results. On the RapidAPI platform used in this study, Web APIs are organized into multiple categories with clear labels, such as *Business*, *Communication*, *Text Analysis*, and etc. These categories provide a coarse-grained indication of the Web service's function. Therefore, identifying the most relevant categories in the early stages of service discovery plays a critical role. Based on this observation, we design a category-level prompt structure that integrates the category list, development scenario, and user query. For example, given the user query: "User diet plan by day", the category-level service discovery prompt can be designed as follows: "*The category list of RapidAPI includes: Text Analysis, Translation, Email, Image Processing, ... Infer the Top-5 most relevant categories from the list based on the following query, considering the potential development scenario and user requirements. For each selected category, briefly describe its relevance to the query. Query: User diet plan by day.*" The category-level prompt process is shown in Fig. 2.

This prompt structure not only guides the LLM in performing deep semantic modeling around the user's queries, but also provides a reliable list of candidate categories for the subsequent service discovery at the API and endpoint level. As a result, it substantially narrows the search space, thus improving both the efficiency and accuracy of the overall service discovery process.

4.2 Service Discovery in API Level

After finishing the category-level service discovery, the process advances to a more fine-grained stage, i.e., API-level service discovery. At this stage, the goal is to further identify specific APIs that match the user's query, from the candidate categories filtered by the category-level service discovery. Since each category may include many APIs with similar functions but different in crucial details, effectively discovering the most relevant APIs becomes a critical challenge. To address this issue, we design an API-level prompt structure. Specifically, we first iterate over the filtered candidate categories. For each category, we first provide the associated API information as input to the LLM to construct a contextual foundation. Then, we prompt the model to further discover which APIs under the given category are directly or indirectly related to the user query. While the API-level service discovery relies on the correctness of category-level, in practice, the number of categories is relatively small, e.g., the RapidAPI dataset contains just over 40 categories. This limited category space, combined with the strong natural language

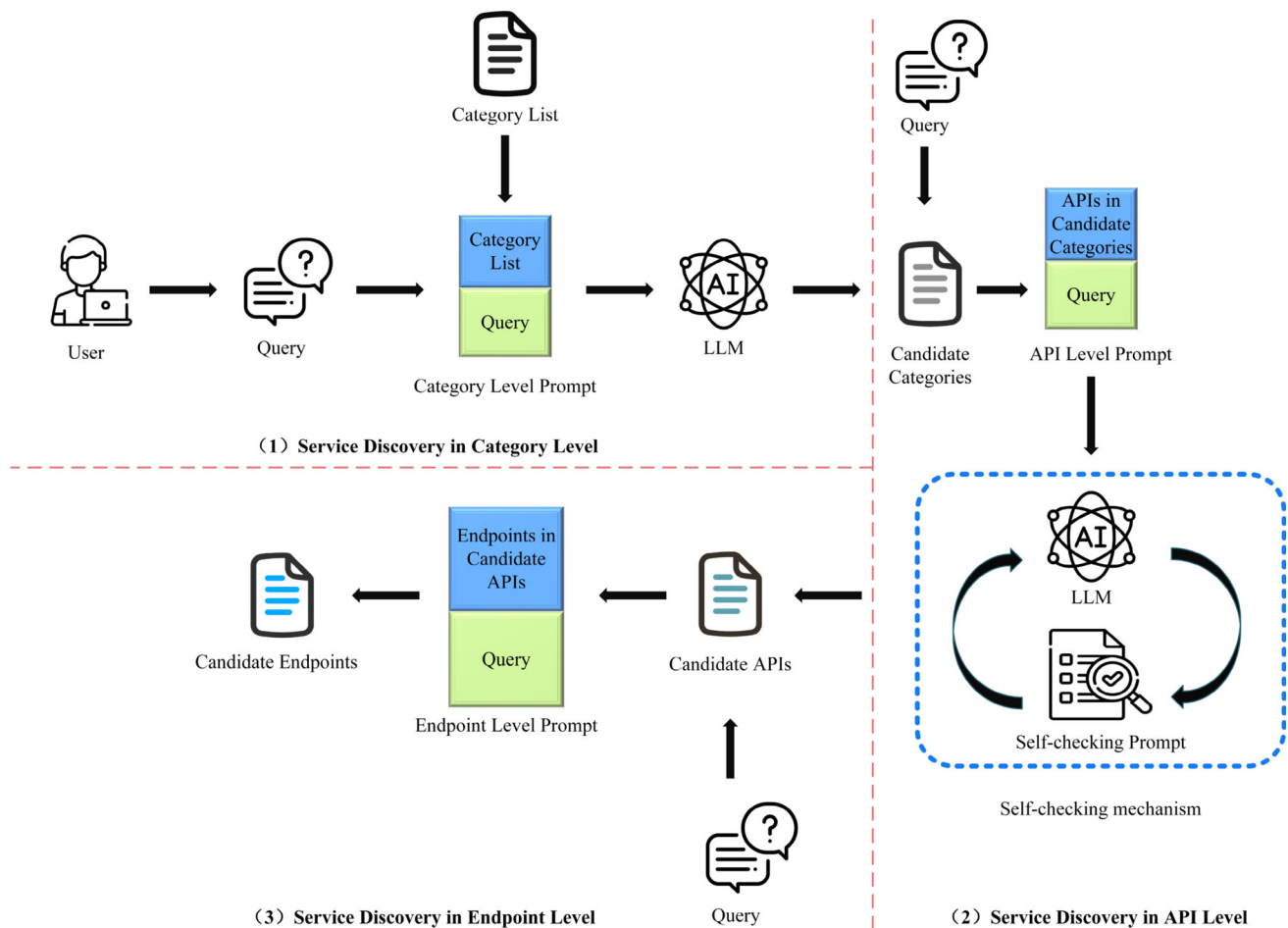


Fig. 1 The LLM-based Web API discovery framework

understanding capabilities of LLMs, ensures accurate category identification, as confirmed by our experiments. Given the user query “User diet plan by day”, if the category-level service discovery stage has discovered “Food” as one of the candidate categories, the first step service discovery prompt at this stage can be designed as follows: “*Please discover the APIs under the ‘Food’ category. List Top-5 APIs that are directly or indirectly related to the query ‘User diet plan by day’. For each API, assign an initial relevance score (0-100) and briefly explain the rationale.*” This prompt structure enables an effective transition from the coarse-grained category level to the fine-grained API-level discovery. The first step of the API-level prompt process is shown in Fig. 3.

However, the API descriptions on the RapidAPI platform are often too general because each API includes many endpoints with different and specific operations. These general descriptions can make it hard for the LLM to find the relevant services accurately. To solve this problem, this paper introduces a self-checking mechanism at the API level. The core idea of this mechanism is to design a prompt structure that guides the LLM to answer two questions for each

candidate API, and accordingly revise its original relevance score. The self-checking mechanism primarily refines the ranking of relevant services (improving MRR) rather than introducing new hits. This is because it reassesses relevance scores without altering the candidate set. The self-checking prompt process consists of the direct relevance assessment and the indirect relevance assessment. Specifically, taking the query “User diet plan by day” as an example, the second step prompt (self-checking) at this stage is designed as follows: “*For each candidate API, perform a self-checking using the following two questions. Adjust the relevance score and briefly explain the rationale behind the adjustment. Q1: Does this API directly support the query ‘User diet plan by day’? (Yes: add 20 points; No: subtract 10 points) Q2: Can this API indirectly assist the task ‘User diet plan by day’? (Yes: add 10 points; No: no change)*” The second step of the API-level prompt process (self-checking) is shown in Fig. 4.

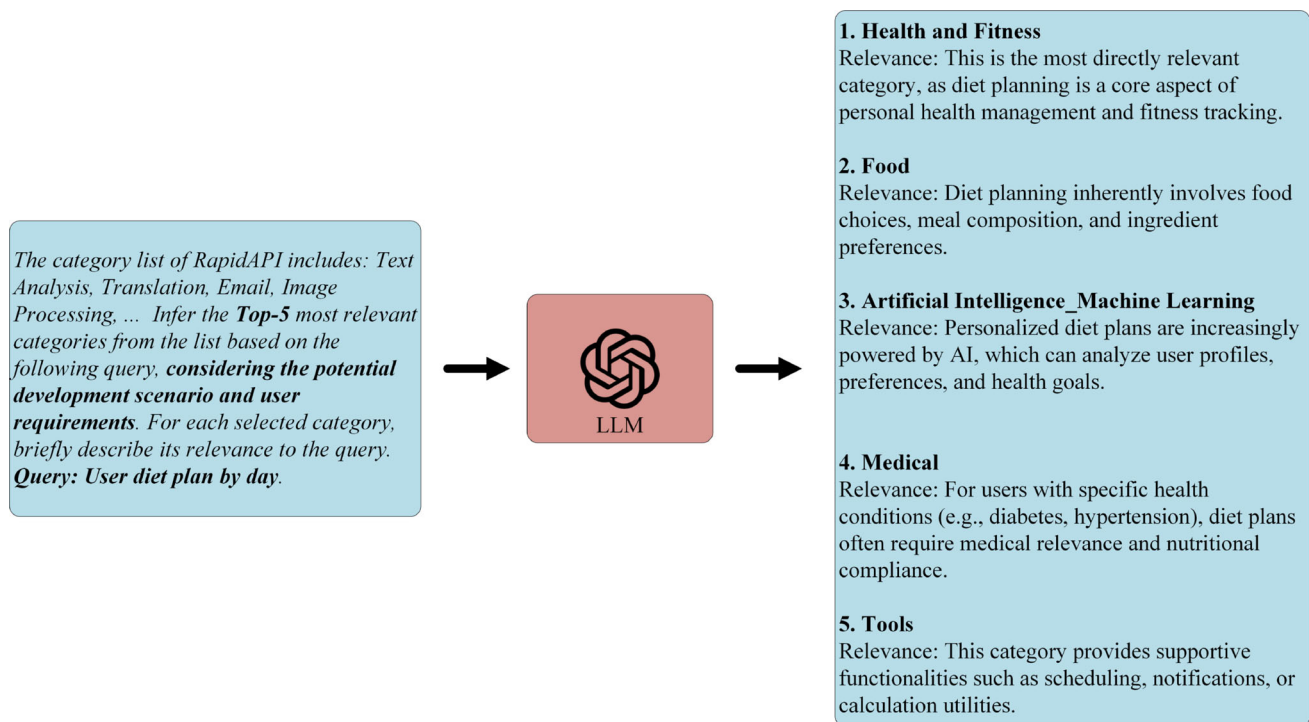


Fig. 2 An example of category-level prompt process

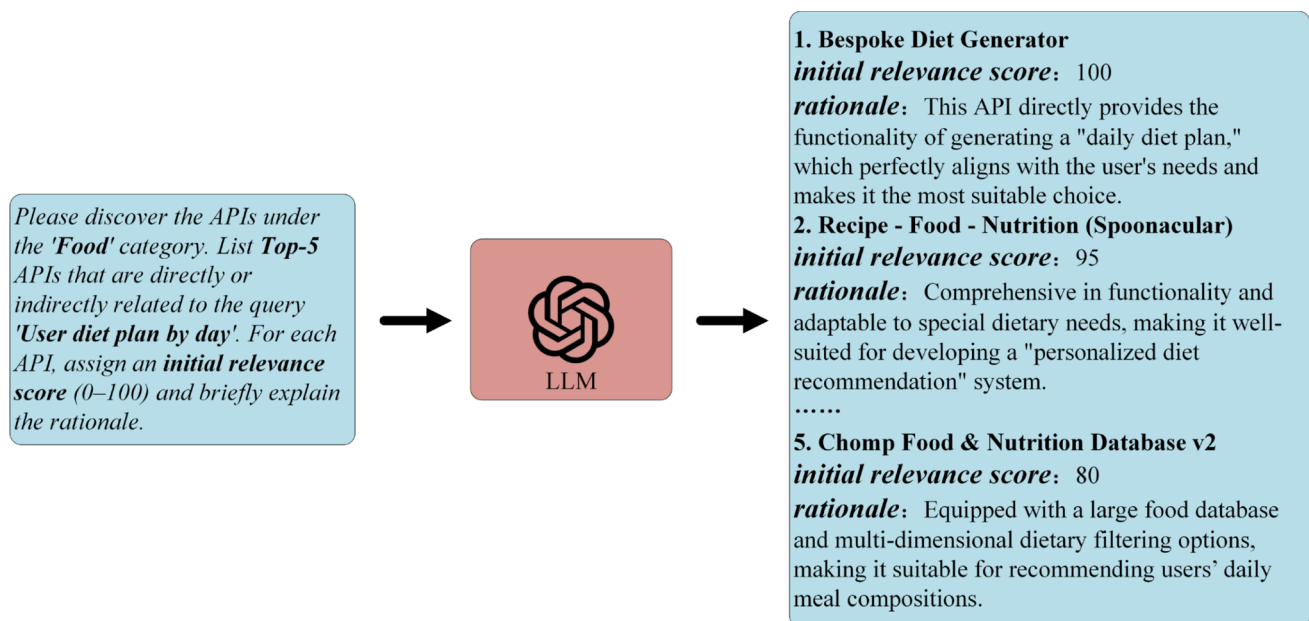


Fig. 3 An example of API-level prompt process

4.3 Service Discovery in Endpoint Level

In the invocation process of the Web API on the RapidAPI platform, the endpoint is the final entry point for a service. Endpoint-level service discovery not only requires semantic relevance with user queries, but also guarantees the invocability of the queried Web APIs. This includes identifying accurate details such as the request URL, required parameters,

and the format of the returned results. These elements are critical, as they often provide essential clues that significantly improve the accuracy of the API discovery. For each candidate API, we first provide the associated Endpoint information as input to the LLM to construct a contextual foundation. Then, we design an endpoint-level prompt structure, where all available endpoints within the candidate APIs are included as the search space. The LLM is asked to dis-

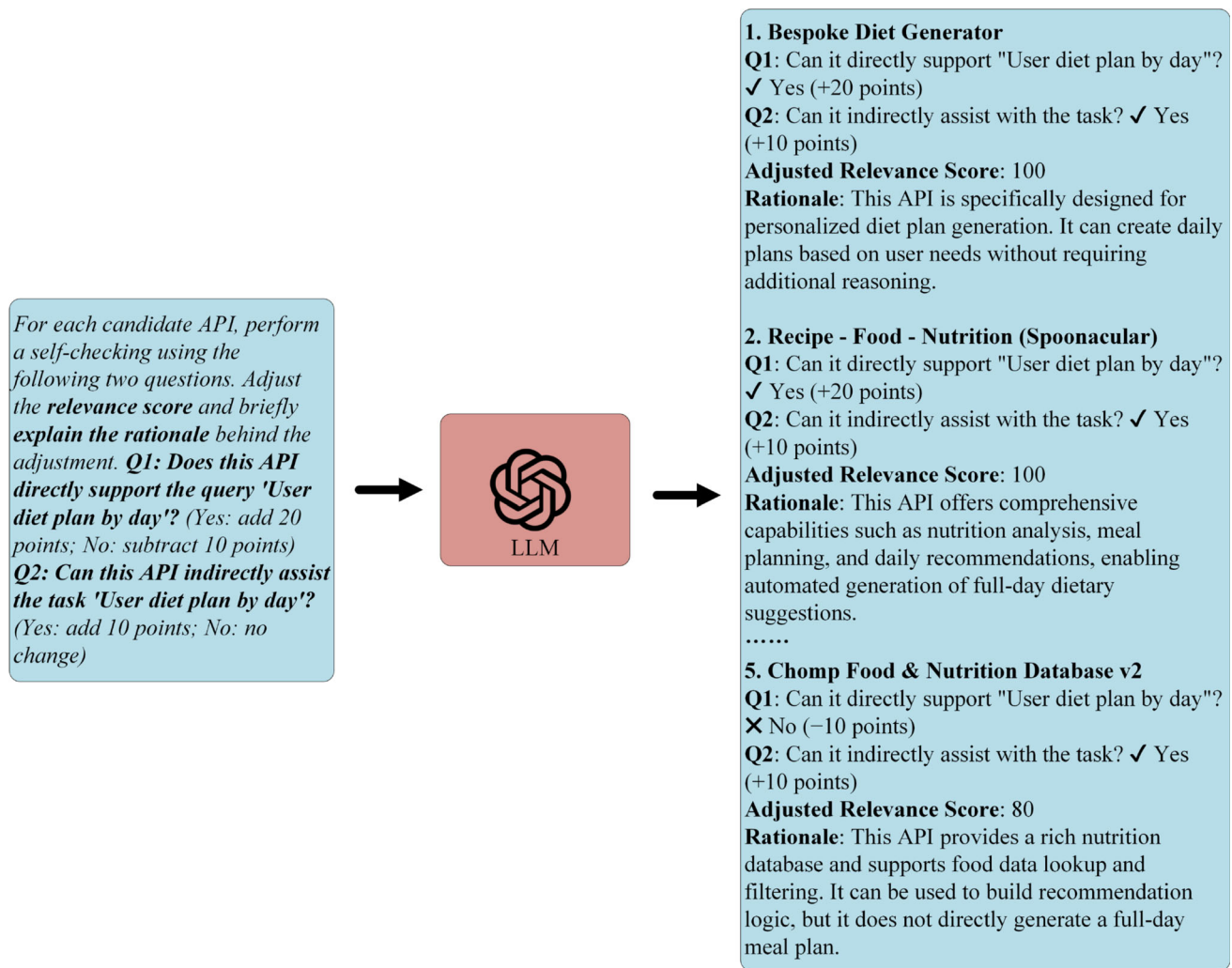


Fig. 4 An example of API-level self-checking prompt process

cover the relevant endpoints considering the user's query and the information of each endpoint. For example, based on the query "User diet plan by day" and a candidate API named "Bespoke Diet Generator", the endpoint-level prompt can be designed as follows: "From the list of endpoints under *Bespoke Diet Generator*, identify the Top-3 endpoints that are most relevant to the user query ("User diet plan by day"), considering the HTTP request type, parameter structure, and functional description. Then, rank these endpoints from most to least relevant, explain the rationale for the ranking, and provide the corresponding links.." The designed endpoint-level prompt process is shown in Fig. 5.

5 Experiments

This section begins with a detailed description of the dataset used in this work, followed by the specification of the eval-

uation metrics. Subsequently, a performance comparison between the proposed method and baseline methods is conducted. We conducted comprehensive experiments and case studies using multiple LLMs to validate the effectiveness and robustness of our method compared with the existing baselines.

5.1 Experimental Setup

Dataset. The dataset used in the experiments is crawled from RapidAPI, one of the largest API marketplace platforms, enabling developers to conveniently access various Web APIs from various providers. The RapidAPI dataset contains 49 categories, approximately 7,000 APIs, and 40,000 endpoints. This dataset covers rich fields, including but not limited to *API Name*, *API Link*, *Required Parameters*, *Optional Parameters*, and *Endpoint Description*. Detailed attribute descriptions of a Web API and examples are provided in

From the list of endpoints under Bespoke Diet Generator, identify the **Top-3** endpoints that are most relevant to the user query ("User diet plan by day"), considering the HTTP request type, parameter structure, and functional description. Then, rank these endpoints from most to least relevant, **explain the rationale for the ranking, and provide the corresponding links.**



1. GET: Get user's diet for a specific day

Relevance Explanation:

Fully matches the requirement of "getting the user's diet plan by day," directly returning the meal plan for a specific day with the highest accuracy.

Invocation Link:

<https://rapidapi.com/genez-io-genez-io-default/api/ bespoke-diet-generator/dietbyday>

2. GET: Get user's diet

Relevance Explanation:

Returns a comprehensive plan including daily meal details. Although it doesn't support per-day requests directly, it can retrieve all days' data at once.

Invocation Link:

<https://rapidapi.com/genez-io-genez-io-default/api/ bespoke-diet-generator/diet>

3. PUT: Generate new diet for user

Relevance Explanation:

While it doesn't directly return the diet plan by day, it is a prerequisite step for obtaining such a plan.

Invocation Link:

<https://rapidapi.com/genez-io-genez-io-default/api/ bespoke-diet-generator/generatediet>

Fig. 5 An example of endpoint-level prompt process

Table 1 Attribute descriptions of a Web API

Field Name	Example
API Name	Text to Speech PRO
API Release Time	2024-02
API Author	Eduard Kleine
API Link	https://rapidapi.com/ptwebsolution/api/text-to-speech-pro/
API Throughput	9.5
API Call Latency	13,840ms
API Service Level	90%
API Host Address	text-to-speech-pro.p.rapidapi.com
Endpoint Name	GET: Get List of Voices
Endpoint Category	Voices
Endpoint Description	Are you looking for an API that can quickly and easily download and parse sitemap.xml files into JSON format?
Required Parameters	[{"parameterName": ["dayIndex", "userId"], "parameterType": ["STRING", "STRING"]}]
Optional Parameters	[{"parameterName": ["group", "part", "user"], "parameterType": ["STRING", "NUMBER", "STRING"]}]

Table 1. The distribution of API categories in the dataset is shown in Fig. 6. Categories over 2% are illustrated individually, while those below 2% are grouped as "The Rest Categories". The figure shows that categories, like *News*,

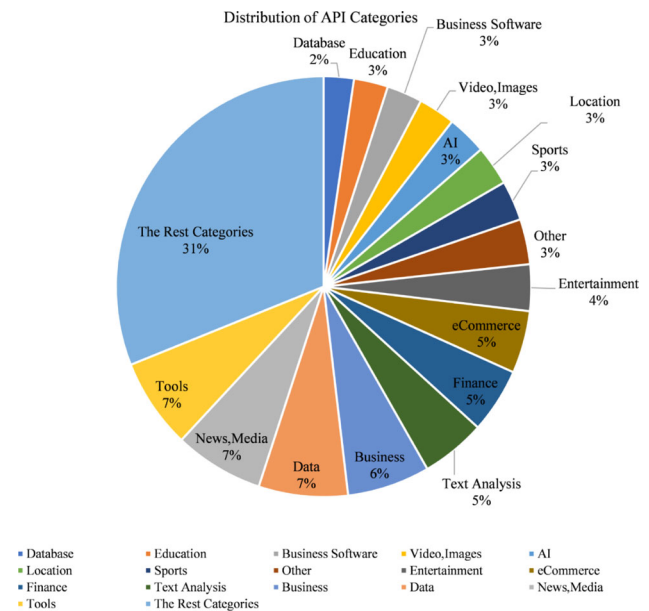


Fig. 6 API categories and their proportions in the dataset

Media, *Tools*, and *Data*, are the ones with the highest proportions.

Queries. To comprehensively evaluate the proposed LLM-based service discovery method in real-world scenarios, we select 10 representative categories from RapidAPI with high API counts and broad application domains, such as *Text Analysis*, *Weather*, and *AI*. From each category, we randomly select 5 endpoints with complete descriptions. Thus, a

Table 2 Keyword extraction examples from endpoint descriptions

Category	Endpoint Function Description	Keywords
AI	Get list of voices	Get list of voices
	Add prompts to queue for image generation. Returned string can be used later to retrieve generated image.	Add prompt to image queue
	Search security by providing the first few words for the security or the company name	Search security info
	Creates an image given a prompt.	Generate image by prompt
	With this endpoint you can check the status for certain job, it requires the job_id.	Check job status
Business	This endpoint returns the information about people associated with the company with the given identifier. The information includes names, roles, email addresses, and phone numbers, if available.	Company people info
	Search for sale listings in a geographical area or around a specific location.	Search sale listings
	Get detailed report for users by day	User daily report
	Get sales history	Sales history
	List properties by listing ID or MLS number	List properties by ID

total of 50 endpoints are selected for the testing data. Due to the lack of service queries from real developers, we simulate the service queries for evaluation. Considering that developers often express their needs concisely, we employ LLMs for user query construction by feeding endpoint descriptions into the model to extract key terms. For example, “*Find regions in a specific country, filtering by optional criteria*” is converted to “*Find regions in country*” by the following prompt: “*Rewrite the following sentence into a concise phrase that summarizes its core meaning. Sentence: Find regions in a specific country, filtering by optional criteria*”. Table 2 illustrates several examples of keyword extraction from endpoint descriptions. These queries are tested over various service discovery methods, including traditional methods and the proposed LLM-based methods.

Metrics. To evaluate the performance of LLM-based service discovery methods, two evaluation metrics are used, i.e., **Hit Rate** (Hit@K) and **Mean Reciprocal Rank** (MRR@K). These metrics are particularly suitable for scenarios where each query corresponds to one correct service, effectively reflecting the model’s accuracy and ranking quality in the top-K list.

Hit@K. Hit@K measures whether the correct service is included in the top-K results. This metric reflects the model’s ability to retrieve relevant services within a limited candidate set. The range of Hit@K is between 0 and 1, with higher values indicating better performance. The equation of Hit@K is defined as Equ. 1, where I is an indicator function that equals 1 if $rank_i \leq K$, and 0 otherwise.

$$Hit@K = \frac{1}{|Q|} \sum_{i=1}^{|Q|} I(rank_i \leq K) \quad (1)$$

MRR@K. MRR@K considers not only whether the correct service is included in the top-K discovery list but also its specific ranking position. For each query, it calculates the reciprocal of the correct service’s rank and averages these values across all queries. Higher MRR values indicate better capability in ranking relevant services in top positions. The equation of MRR@K is defined as Equ. 2.

$$MRR@K = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i} \quad (2)$$

Baseline. Due to the lack of semantic annotation in RESTful APIs, we mainly compare our method with the semantic-based service discovery methods as follows.

- **TextOnly** [23]: This method uses description embeddings [27] and calculates the cosine similarity between the embedding vectors of the user request and each candidate service as the matching score. The discovery results are generated by ranking the candidate services based on these scores. This method is designed to evaluate the impact of interface parameter information on the effectiveness of service discovery.
- **Sentence-BERT** [28]: Sentence-BERT is a BERT-based sentence embedding generation model. Sentence-BERT

captures deeper contextual and semantic relationships between user queries and service descriptions. We employ Sentence-BERT for the embedding of user queries and endpoint descriptions, and compute their cosine similarity for service matching.

- **Ours (no self-checking)**: This method shares the same framework as our proposed method but removes the self-checking mechanism during the process of service discovery at the API level, aiming to validate the effectiveness of the self-checking mechanism in our method.
- **LLM Direct Matching** : In this method, the user query is directly provided to the LLM (e.g., GPT-4, DeepSeek-R1, LLaMA-2) without hierarchical prompting or self-checking. The model is asked to discover the most relevant services based solely on its internal understanding and inference.

5.2 Performance Evaluation

To validate the effectiveness of the proposed method, we conduct experiments using multiple LLMs such as GPT-4, deepseek-R1, Llama-2 and perform comparative evaluations against baseline methods on the RapidAPI dataset. The evaluation spans multiple list lengths (top-K = 5, 10, and 20) to assess both accuracy and ranking quality comprehensively. The specific performance comparison results are illustrated in Table 3. The experimental results reveal distinct performance characteristics across methods. The TextOnly method exhibits the lowest accuracy in service discovery, achieving only 0.4600 under Hit and 0.3220 under MRR at topK=5 because it relies solely on surface-level lexical features without capturing the deeper semantic meaning of queries and service descriptions. Sentence-BERT improves semantic understanding through contextualized sentence embeddings and demonstrates better performance compared to the TextOnly method. However, its static vector matching paradigm remains constrained by an inability to leverage structural service hierarchies. LLM Direct Matching, which directly inputs user queries into LLM (GPT-4, deepseek-R1 and Llama-2) without hierarchical prompting or self-checking, is also evaluated. When top-K=20, LLM Direct Matching-GPT-4 achieves a Hit@20 of 0.700 and an MRR@20 of 0.5863. The results indicate that LLM Direct Matching consistently outperforms simple embedding-based baselines (TextOnly and Sentence-BERT), as LLMs are able to leverage deeper semantic understanding and contextual reasoning, but still fall short of our proposed hierarchical service discovery method.

Our proposed method significantly outperforms baselines across different lengths of the result list. Furthermore, when implemented with different LLMs, such as GPT-4, deepseek-R1 and Llama-2, our method consistently achieves higher Hit@K and MRR@K scores, demonstrating its robustness

and adaptability. Among the three LLMs, GPT-4 delivers the best overall performance. Typically, when top-K=20, our method reaches a Hit@20 of 0.7400 and an MRR@20 of 0.6773. Deepseek-R1 also shows competitive performance, particularly in ranking accuracy, with an MRR@20 of 0.6793. Although our method with Llama-2 is slightly worse compared with GPT-4 and deepseek-R1, it is still better than all the other baselines, confirming the effectiveness of our hierarchical prompt design even with lighter-weight models. Additionally, the comparison between the “no self-checking” variants and full versions of our method reveals that incorporating the self-checking mechanism yields consistent performance improvements. This enhancement is particularly notable in terms of MRR, indicating that the self-checking mechanism is helpful to refine the ranking of relevant APIs, leading to more accurate service discovery results.

5.3 Case Study

Case studies validate the effectiveness of the proposed method, highlighting the potential of LLMs in advancing service discovery. Table 4 illustrates three examples of the service discovery process driven by GPT-4 based on user queries. The entries within the API and endpoint lists are explicitly ordered by their relevance, and the red contents represent the expected correct answers. For each input, the model outputs a structured list of results, including the inferred category list, the API list before and after self-checking, and the endpoint list. For the user query “User diet plan by day”, the model accurately discovers relevant categories, such as *Health*, *Food*, and *Medical*, and outputs the APIs, such as *Bespoke Diet Generator* and *Recipe - Food - Nutrition (Spoonacular)*. After applying the self-checking mechanism, the API list is refined. Obviously, *Recipe Food Nutrition (Spoonacular)*, which provides personalized diet advice and meal plans for users, is retained and ranked higher on the list. Besides, in the endpoint list, *GET: Get user's diet* is correctly identified as the most suitable endpoint, directly satisfying the user's requirement.

Table 5 illustrates the case analysis. The first column of the table shows the actual user queries. The discovered top-5 endpoints of the four methods are shown in the table. The actual endpoint discovered by our method appears in the top position in the three investigated examples. Meanwhile, the ranking results discovered by LLM Direct Matching-GPT-4 outperform those of Sentence-BERT and TextOnly, further demonstrating the effectiveness of leveraging the deep semantic understanding capabilities of large language models for service discovery compared with traditional methods. Overall, our method can obtain a more accurate list of candidate endpoints and retrieve the required endpoints in

Table 3 Performance comparison of different service discovery methods

Top-K	Method	Hit@K	MRR@K
5	TextOnly	0.4600	0.3220
	Sentence-BERT	0.4800	0.3503
	LLM Direct Matching-GPT-4	0.5200	0.4263
	LLM Direct Matching-deepseek-R1	0.5400	0.4433
	LLM Direct Matching-Llama-2	0.4800	0.3729
	Ours-GPT-4 (no self-checking)	0.6000	0.4891
	Ours-GPT-4	0.6000	0.5073
	Ours-deepseek-R1 (no self-checking)	0.5800	0.4795
	Ours-deepseek-R1	0.5800	0.4893
	Ours-Llama-2 (no self-checking)	0.5200	0.4505
	Ours-Llama-2	0.5200	0.4591
10	TextOnly	0.5800	0.3591
	Sentence-BERT	0.5600	0.3609
	LLM Direct Matching-GPT-4	0.6400	0.5200
	LLM Direct Matching-deepseek-R1	0.6200	0.5031
	LLM Direct Matching-Llama-2	0.5800	0.4679
	Ours-GPT-4 (no self-checking)	0.6600	0.5491
	Ours-GPT-4	0.6600	0.5673
	Ours-deepseek-R1 (no self-checking)	0.6600	0.5579
	Ours-deepseek-R1	0.6600	0.5673
	Ours-Llama-2 (no self-checking)	0.6200	0.5011
	Ours-Llama-2	0.6200	0.5121
20	TextOnly	0.6400	0.3638
	Sentence-BERT	0.6400	0.3668
	LLM Direct Matching-GPT-4	0.7000	0.5863
	LLM Direct Matching-deepseek-R1	0.6800	0.5733
	LLM Direct Matching-Llama-2	0.6300	0.5129
	Ours-GPT-4 (no self-checking)	0.7400	0.6632
	Ours-GPT-4	0.7400	0.6773
	Ours-deepseek-R1 (no self-checking)	0.7200	0.6522
	Ours-deepseek-R1	0.7200	0.6793
	Ours-Llama-2 (no self-checking)	0.6400	0.5534
	Ours-Llama-2	0.6400	0.5723

the top positions, which shows the effectiveness of the LLM-based framework for Web API service discovery.

5.4 Discussion

The experimental results presented in this study compellingly demonstrate the efficacy of the proposed LLM-based hierarchical framework in advancing the state-of-the-art in RESTful Web API discovery. Our framework, through its hierarchical prompting strategy, explicitly models this structure, guiding the LLM to first understand the broad functional domain (category), then identify potential solution providers (API), and finally pinpoint the exact operational unit (endpoint). This stepwise refinement mirrors the natural cognitive

process of a developer, thereby leading to more intuitive and accurate results.

While the proposed framework demonstrates promising results, several aspects warrant further discussion or clarification. First, while RapidAPI is chosen as the primary dataset due to its large scale and standardized documentation, we acknowledge that different datasets may have varying organizational structures. Applying our framework to other datasets (e.g., ProgrammableWeb) may require adaptations or simplifications, such as reducing the hierarchy to two levels. Second, although our self-checking mechanism alleviates the hallucination problem of large language models (LLMs) by verifying the relevance of candidate APIs and endpoints, it cannot eliminate hallucinations entirely.

Table 4 Example results of LLM-based service discovery method with GPT-4

Request input	Category List	API List	API List (After Self-checking)	Endpoint List
User diet plan by day	1. Health 2. Food vs 3. Medical	1. Bespoke Diet Generator 2. Recipe Food Nutrition (Spoonacular) 3. EdamamFood and GroceryDatabase 4. Chomp Food ...	1. Recipe Food Nutrition (Spoonacular) 2. Bespoke Diet Generator 3. EdamamFood and GroceryDatabase 4. Chomp Food ...	1. GET: Get user's diet 2. GET: Get user's diet for a specific day 3. GET: Retrieve user's food preferences 4. PUT: Set user's food preferences 5. PUT: Generate new diet for user 6. GET: Get food replacement options in diet ...
Search free videos	1. Video 2. Media 3. Search	1. Petey Vid Video Search API 2. Bing Video Search 3. Pexels 4. Youtube Search All Media 5. Downloader ...	1. Petey Vid Video Search API 2. Pexels 3. Youtube Search All Media 4. Bing Video Search 5. Downloader ...	1. GET: Search for Videos 2. GET: Curated Photos 3. GET: Get a Video by ID 4. GET: Search for Photos 5. GET: Popular Videos 6. GET: Get a Photo by ID 7. GET: New mlem ...
Find regions in country	1. Location 2. Data 3. Mapping	1. Geography 2. GeoDB Cities 3. GeoDB CitiesGraphQL 4. Maps Data 5. autocompletestr ...	1. GeoDB Cities 2. Geography 3. GeoDB CitiesGraphQL 4. Maps Data 5. autocompletestr ...	1. GET: Countries 2. GET: Country Details 3. GET: Country Regions 4. GET: Country Places 5. GET: Country Region Details 6. GET: Places 7. GET: Places Near Location 8. GET: Country Region Places 9. GET: Cities ...

Table 5 Case study

Request input	TextOnly	Sentence-BERT	LLM Direct Matching-GPT-4	Ours-GPT-4
User diet plan by day	1. PUT: Generate new diet for user 2. GET: Get user's diet for a specific day 3. DELETE: Clear Meal Plan Day 4. GET: Get user's diet 5. POST: Add Meal Plan Template ...	1. GET: Get user's diet for a specific day 2. GET: Get user's diet 3. POST: Add Meal Plan Template 4. GET: Generate Meal Plan 5. POST: Generate Shopping List ...	* 1. GET: Get user's diet for a specific day 2. GET: Get user's diet 3. PUT: Generate new diet for user 3. GET: Retrieve user's food preferences 5. POST: Generate Shopping List ...	1. GET: Get user's diet 2. GET: Get user's diet for a specific day 3. GET: Retrieve user's food preferences 4. PUT: Set user's food preferences 5. PUT: Generate new diet for user ...
Search free videos	1. POST: getVideos In Group 2. POST: Fast SD API 3. POST: Retrieve 4. GET: Minimum Rating 5. GET: GET 10 objects in result ...	1. POST: getVideos In Group 2. POST: checkHash 3. GET: GET 10 objects in result 4. GET: Render a URL as an image or PDF 5. POST: Fast SD API ...	1. GET: Get a Video by ID 2. POST: checkHash 3. GET: Search for Videos 4. GET: Popular Videos 5. POST: getVideos In Group ...	1. GET: Search for Videos 2. GET: Curated Photos 3. GET: Get a Video by ID 4. GET: Search for Photos 5. GET: Popular Videos ...
Find regions in country	1. GET: Latest news 2. POST: Current vimshot-tari dasha 3. GET: Country Regions 4. GET: Country Details 5. GET: Houseplants by Family ...	1. GET: Latest news 2. GET: Country Region Places 3. GET: Country Regions 4. GET: Country Places 5. GET: Country Region Details ...	1. GET: Country Region Details 2. GET: Country Details 3. GET: Country Places 4. GET: Country Regions 5. GET: Countries ...	1. GET: Countries 2. GET: Country Details 3. GET: Country Regions 4. GET: Country Places 5. GET: Country Region Details ...

However, by cross-validating with the query context and prompting the LLM to provide explicit reasoning during self-checking, hallucinations can often be effectively and more easily identified by users. Finally, multi-step LLM invocations introduce additional latency and cost, especially when using models such as GPT-4. In our experiments, we employ GPT-4, DeepSeek-R1, and LLaMA-2 as representative large models. For reproducibility, the maximum input token limits are provided as examples: GPT-4 supports up to 8K or 32K tokens depending on the version, DeepSeek-R1 allows up to 16K tokens, and LLaMA-2 typically supports 4K to 8K tokens. While accuracy improvement is our primary focus, we recognize the importance of efficiency for practical adoption. To reduce cost, technical optimizations such as caching category-level results and employing lightweight models in earlier steps can be applied.

6 Conclusion

The rapid proliferation of RESTful Web APIs has created both opportunities and challenges for developers seeking to discover and integrate relevant services efficiently. In this paper, we propose a novel LLM-based framework for Web API service discovery, specifically designed for the RapidAPI platform. Our approach leverages large language models to bridge the gap between user queries and API documentation through a three-level prompt strategy, i.e., category level, API level, and endpoint level, progressively narrowing the search space to identify the most relevant APIs and endpoints. Additionally, we introduced a self-checking mechanism to enhance the accuracy and reliability of the discovery process. Extensive experiments conducted across diverse development scenarios demonstrated the superiority of our method over baselines in terms of both Hit@K and MRR@K metrics. The results highlighted the robustness and adaptability of our framework when implemented with different LLMs, such as GPT-4, deepseek-R1, and Llama-2. Case studies further validated the effectiveness of our method, showcasing its ability to accurately identify relevant categories, APIs, and endpoints based on user queries. Future work will include extending the framework to support multi-lingual queries, integrating real-time feedback mechanisms for continuous improvement, and exploring the application of our hierarchical prompt strategy in other software engineering tasks.

Funding This work is partially supported by Educational Commission of Hunan Province of China under Grant No: 23A0359, and Hunan Provincial Natural Science Foundation Project under Grant No: 2022JJ30262.

Declarations

Conflicts of Interest The authors declare no conflict of interest.

References

1. Kang G, Ren H, Chen W, Liu J, Cao B, Xu Y (2025) Web API recommendation via exploring textual and structural semantics with contrastive learning and joint training. *IEEE Transactions on Network and Service Management* 22(2):1558–1568 (IEEE)
2. Kang G, Liang B, Liu J, Wen Y, Xiao Y, Nie H (2024) Web API recommendation via leveraging content and network semantics. *IEEE Transactions on Network and Service Management* 21(6):5977–5991 (IEEE)
3. Chen F, Lu C, Wu H, Li M (2017) A semantic similarity measure integrating multiple conceptual relationships for Web service discovery. *Expert Systems with Applications* 67(2017):19–31 (Elsevier)
4. Richardson L, Amundsen M, Ruby S (2013) RESTful Web APIs: Services for a Changing World, O'Reilly
5. Segura S, Parejo J.A, Troya J, Ruiz-Cortés A (2018) Metamorphic testing of RESTful Web APIs. In: International Conference on Software Engineering, pp. 882–882 . ACM
6. Li W, Ren H, Nie Y, Liu Z, Kang G, Liu J, Peng Z (2024) Crawling and exploring RESTful Web APIs from RapidAPI. In: International Conference on Service Science, pp. 88–103 . Springer
7. Martin-Lopez A, Segura S, Ruiz-Cortés A (2022) Online testing of RESTful APIs: Promises and challenges. In: ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 408–420 . ACM
8. Azmy MR, Muhamad W, *et al* (2020) Advanced technologies to support service discovery in service-oriented systems. In: International Conference on Information Technology Systems and Innovation, pp. 300–305 . IEEE
9. Tokmak AV, Akbulut A, Catal C (2024) Web service discovery: Rationale, challenges, and solution directions. *Computer Standards & Interfaces* 88(2024):5977–5991 (Elsevier)
10. Zhou J, Zhang T, Meng H, Xiao L, Chen G, Li D (2008) Web service discovery based on keyword clustering and ontology. In: IEEE International Conference on Granular Computing, pp. 844–848 . IEEE
11. Liu L, Bahrami M, Park J, Chen WP (2020) Web API search: discover Web API and its endpoint with natural language queries. In: International Conference on Web Services, pp. 96–113 . Springer
12. Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman FL, Almeida D, Altenschmidt J, Altman S, Anadkat S, *et al* (2023) GPT-4 technical report. arXiv preprint [arXiv:2303.08774](https://arxiv.org/abs/2303.08774)
13. Naveed H, Khan AU, Qiu S, Saqib M, Anwar S, Usman M, Akhtar N, Barnes N, Mian A (2023) A comprehensive overview of large language models. arXiv preprint [arXiv:2307.06435](https://arxiv.org/abs/2307.06435)
14. Kumar GS, Lakshmi C (2016) A literature survey on Web service discovery. *International Journal of Engineering and Technology* 8(2016):1045–1051
15. Obidallah WJ, Raaheem B (2017) A survey on Web service discovery approaches. In: International Conference on Internet of Things, Data and Cloud Computing, pp. 1–8 . Springer
16. Zhang Y, Zheng Z, Lyu MR (2010) Wsexpress: A QoS-aware search engine for Web services. In: 2010 IEEE International Conference on Web Services, pp. 91–98 . IEEE

17. Martin D, Burstein M, McDermott D, McIlraith S, Paolucci M, Sycara K, McGuinness DL, Sirin E, Srinivasan N (2007) Bringing semantics to Web services with OWL-S. *World Wide Web* 10(2007):243–277 (**Springer**)
18. Paolucci M, Kawamura T, Payne TR, Sycara K (2002) Semantic matching of Web services capabilities. In: *International Semantic Web Conference*, pp. 333–347. Springer
19. Taieb MAH, Aouicha MB, Hamadou AB (2014) Ontology-based approach for measuring semantic similarity. *Engineering Applications of Artificial Intelligence* 36(2014):238–261 (**Elsevier**)
20. Zhang X, Liu J, Cao B, Xiao Q, Wen Y (2019) Web service discovery based on information gain theory and bilstm with attention mechanism. In: *International Conference on Collaborative Computing: Networking, Applications and Worksharing*, pp. 643–658. Springer
21. He Q, Zhou R, Zhang X, Wang Y, Ye D, Chen F, Grundy JC, Yang Y (2016) Keyword search for building service-based systems. *IEEE Transactions on Software Engineering* 43(7):658–674 (**IEEE**)
22. Ma S-P, Chen Y-J, Syu Y, Lin H-J, Fanjiang Y-Y (2018) Test-oriented RESTful service discovery with semantic interface compatibility. *IEEE Transactions on Services Computing* 14(5):1571–1584 (**IEEE**)
23. Wang S, Zhou Y, Ding Z (2022) Automated RESTful API service discovery with various interface features. In: *International Conference on Service-Oriented Computing*, pp. 54–70. Springer
24. Wei C, Fan Y, Zhang J (2022) High-order social graph neural network for service recommendation. *IEEE Trans Netw Serv Manage* 19(4):4615–4628
25. Liu H, et al (2024) Large language model aided QoS prediction for service recommendation. *arXiv preprint* [arXiv:2408.02223](https://arxiv.org/abs/2408.02223)
26. Zou G et al (2025) Llm-enhanced service semantic representation and category co-occurrence feature augmentation for Web API recommendation. *Information Processing & Management* 62(6):104219
27. Gao T, Yao X, Chen D (2021) Simcse: Simple contrastive learning of sentence embeddings. *arXiv preprint* [arXiv:2104.08821](https://arxiv.org/abs/2104.08821)
28. Reimers N, Gurevych I (2019) Sentence-BERT: Sentence embeddings using siamese BERT-networks. *arXiv preprint* [arXiv:1908.10084](https://arxiv.org/abs/1908.10084)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com